

Guía paso a paso: modelo de caja, Flexbox y Grid en tu portafolio

Curso Diseño Web · Centro de Informática USS · Protech XP · Sesión 10

Las medidas de la Disposición automática de Figma pasan a CSS: el Hero Section con Flexbox y las páginas interiores a dos columnas con Flexbox y Grid.

Vienes de la sesión 09: necesitas `global.css`, `header.css` y `contenido.css` funcionando y enlazados. Si te falta algo, compáralo con la carpeta resuelta de la sesión 09 antes de empezar. Hoy también construyes el Hero Section, que estaba previsto para la sesión 08 (feriado).

Qué vas a hacer

Hasta ahora tu CSS decía cómo se ve cada cosa: color, letra, bordes. Hoy decide dónde va. Primero miras el modelo de caja en DevTools; después construyes el Hero de la portada con Flexbox y, por último, pasas las páginas interiores a las dos columnas de tu diseño con Flexbox y Grid.

Archivo	Qué cambia hoy	Capas en Figma
<code>index.html</code>	El marcado del Hero dentro de <code><main></code> y el enlace a <code>index.css</code>	<code>hero-left</code> y <code>hero-right</code>
<code>index.css (nuevo)</code>	El Hero a dos columnas, los botones y la tarjeta de código	<code>hero-section</code> (Pasos 6 al 9)
<code>header.css</code>	El pie de página pasa a Flexbox en columna	<code>footer-wrapper</code> (Paso 13)
<code>contenido.css</code>	La caja de 1440; Sobre mí y Contacto a dos columnas con Flexbox; Proyectos y el formulario en rejilla con Grid	<code>about-content</code> , <code>projects-grid</code> , <code>contact-content</code>
Las tres páginas interiores	Clases que conectan cada bloque con su capa de Figma	—
<code>assets/img/</code>	Dos imágenes de proyecto de 628 × 220	<code>project-image</code> (Paso 14)

Cómo se reparte

Momento	Pasos	Qué practicas
En clase, juntos	0 al 3	El modelo de caja en DevTools; Flexbox en el Hero; el pie en columna
En clase, tú	4 y 5	Grid en Proyectos; Flexbox en Sobre mí
Tarea	6 y 7	Contacto con Flexbox y el formulario con Grid; la multimedia en rejilla; validar y publicar
En Figma	Guía de Figma	Pasos 12 al 14 del manual: Contacto, pie de página e imágenes

Material del paquete

- **mi-portafolio-dagner/**: la carpeta resuelta. Trae las dos imágenes de proyecto en `assets/img/`.

- **Guía de Figma • Sesión 10:** los Pasos 12 al 14 del manual, clic por clic.
- **Tu archivo de Figma:** con la cabecera, el Hero, Sobre mí y Proyectos terminados (Pasos 1 al 11).
- **Esta guía:** cada paso indica qué archivo tocar, qué código escribir y cómo comprobarlo.

Antes de escribir: la Disposición automática es Flexbox

Cada marco de Figma con **Disposición automática** se escribe en CSS con `display: flex`. Selecciona el marco, mira el panel derecho y traduce cada control con esta tabla:

En Figma	En CSS	Ejemplo del portafolio
Disposición automática (Shift + A)	<code>display: flex</code>	<code>header, .hero-section</code>
Flujo: flecha derecha o flecha abajo	<code>flex-direction: row o column</code>	<code>.hero-left { flex-direction: column; }</code>
Espacio (separación entre hijos)	<code>gap</code>	<code>gap: 64px;</code> en <code>.hero-section</code>
Espacio: Auto	<code>justify-content: space-between</code>	<code>header, .code-header</code>
Espaciado (relleno interno)	<code>padding</code>	<code>padding: 160px 80px 100px 80px;</code>
Alineación	<code>align-items</code> (y <code>justify-content</code>)	<code>align-items: center;</code>
Ancho fijo • Altura fija	<code>width • height</code> (en un ítem flex, <code>flex: 0 0 480px</code>)	<code>.about-content aside</code>
Llenar el contenedor	<code>flex: 1</code> (o <code>width: 100%</code>)	<code>.hero-left { flex: 1; }</code>
Ajustar al contenido	No necesita propiedad	Los botones del Hero
Recortar contenido	<code>overflow: hidden</code>	<code>.code-card, .project-card</code>
Relleno de tipo Imagen, modo Llenar	<code>object-fit: cover</code>	<code>.project-image</code>

¿Y Grid? en Figma, `projects-grid` es un Flujo horizontal con dos tarjetas en Llenar el contenedor. En CSS ese resultado se consigue con Flexbox o con Grid. Usamos Grid cuando las piezas forman filas y columnas que deben medir lo mismo, como una cuadrícula de tarjetas o un formulario a dos columnas.

Si no coinciden, manda Figma: cuando una medida de tu CSS no sea igual a la de tu diseño, corrige el CSS.

Paso 0 • El modelo de caja en DevTools

Toda etiqueta es una caja con cuatro capas, de adentro hacia afuera: el **contenido**, el **padding** (relleno), el **border** (borde) y el **margin** (margen). Antes de maquetar, míralo en tu página:

1. Abre `sobre-mi.html` en el navegador, haz clic derecho sobre el texto y elige **Inspeccionar**.
2. En el panel de elementos, haz clic en la etiqueta `<main>`.
3. Abre la pestaña **Computed**. Arriba está el diagrama de la caja: el contenido en el centro, el padding de 24 a los lados y 100 arriba y abajo.

4. Pasa el cursor por cada capa del diagrama: el navegador la pinta sobre la página.

Ahora prueba qué hace la regla `box-sizing: border-box` que escribiste en `global.css`:

1. Selecciona `<main>`. En la pestaña **Styles**, busca la regla `*`, `*::before`, `*::after`.
2. Desmarca la casilla de `box-sizing: border-box`. La caja pasa de 800 a 848 px de ancho: el padding se suma por fuera.
3. Vuelve a marcarla. Con `border-box`, el ancho que escribes incluye el padding y el borde: 800 son 800.

Propiedad	Qué hace	Dónde lo usas
display: block	La caja ocupa todo el ancho y empieza en una línea nueva. Así son <code>div</code> , <code>p</code> , <code>section</code>	<code>.project-image</code> , <code>.contact-card</code> a
display: inline	La caja va dentro de la línea de texto y no acepta <code>width</code> , <code>height</code> ni padding vertical. Así son <code>a</code> , <code>span</code> , <code>img</code>	Los enlaces del menú
display: inline-block	Va en la línea, pero sí acepta <code>width</code> y <code>height</code>	Los tres círculos <code>.dot</code>
display: flex · grid	La caja ordena a sus hijos en fila, en columna o en rejilla	Hoy, en todo el portafolio
margin: 0 auto	Reparte el espacio sobrante a los lados: centra una caja con ancho máximo	<code>main</code> , <code>.hero-section</code>
max-width	La caja crece hasta ese ancho, pero puede encogerse	<code>main</code> , <code>.hero-left</code>

Paso 1 · Maqueta el Hero Section en `index.html`

Abre `index.html`. En el `<head>`, debajo de los dos enlaces de la sesión 09, añade la hoja propia de la portada:

```
<!-- 1. Tokens y Reset Global -->
<link rel="stylesheet" href="assets/css/global.css" />
<!-- 2. Componentes comunes (Header / Footer) -->
<link rel="stylesheet" href="assets/css/header.css" />
<!-- 3. Estilos específicos de esta página (Hero) -->
<link rel="stylesheet" href="assets/css/index.css" />
```

Dentro de la etiqueta `<main>`, escribe la sección de presentación con sus dos columnas (`hero-left` y `hero-right`):

```
<main>
  <!-- PRESENTACIÓN · Figma: hero-section -->
  <section class="hero-section">
    <!-- Columna Izquierda · Figma: hero-left -->
    <div class="hero-left">
      <h2 class="hero-title">
        Formando a la próxima generación de desarrolladores web.
      </h2>
      <p class="hero-description">
        Docente e Ingeniero enfocado en el desarrollo web moderno,
        arquitectura de software y formación tecnológica integral en el
        Centro de Informática USS.
      </p>
      <!-- Botones de Acción · Figma: hero-actions -->
      <div class="hero-actions">
        <a href="mis-proyectos.html" class="btn btn-primary">Ver Proyectos</a>
```

```

        <a href="contactame.html" class="btn btn-secondary">Contáctame</a>
      </div>
    </div>

    <!-- Columna Derecha · Figma: hero-right (Tarjeta de código) -->
    <div class="hero-right">
      <div class="code-card">
        <div class="code-header">
          <div class="window-controls">
            <span class="dot dot-red"></span>
            <span class="dot dot-yellow"></span>
            <span class="dot dot-green"></span>
          </div>
          <span class="code-filename">dagner-chuman.ts</span>
        </div>
        <pre class="code-body"><code>const docente = {
nombre: "Dagner Chuman",
rol: "Docente & Ingeniero Web",
institucion: "Centro de Informática USS",
skills: ["HTML5", "CSS3", "JavaScript"],
mision: "Transformar ideas en código"
};</code></pre>
      </div>
    </div>
  </section>
</main>

```

Detalle semántico: cada página lleva un solo `<h1>`, y el tuyo ya está en la cabecera: es tu nombre. Por eso el título del Hero es un `<h2>`; su tamaño lo decide la clase `hero-title`, no la etiqueta. Usamos `<pre><code>` para respetar exactamente los espacios del código, y el signo `&` se escribe `&`.

Paso 2 · El Hero con Flexbox (assets/css/index.css)

Un contenedor flex tiene dos ejes. El **eje principal** sigue el Flujo: horizontal con `row`, vertical con `column`. El **eje cruzado** es el otro. `justify-content` reparte los hijos en el eje principal y `align-items` los alinea en el cruzado.

Propiedad	Se escribe en	Qué hace
display: flex	El contenedor	Sus hijos directos pasan a ser ítems flex y se colocan en fila
flex-direction	El contenedor	<code>row</code> (fila, por defecto) o <code>column</code> (columna)
gap	El contenedor	La separación entre ítems: el Espacio de Figma
justify-content	El contenedor	Reparte en el eje principal: <code>flex-start</code> , <code>center</code> , <code>space-between</code> ...
align-items	El contenedor	Alinea en el eje cruzado: <code>stretch</code> (por defecto), <code>center</code> , <code>flex-start</code> ...
flex-wrap: wrap	El contenedor	Si no caben, los ítems bajan a otra línea
flex: 1	Un ítem	El ítem crece para tomar el espacio que sobra: Llenar el contenedor
flex: 0 0 480px	Un ítem	No crece, no se encoge y mide 480: Ancho fijo

En Figma: las medidas del Hero

Capa (paso del manual)	En Figma	En index.css
hero-section (Paso 6)	Flujo horizontal; Espacio 64; Espaciado 160 · 80 · 100 · 80	display: flex; · gap: 64px; · padding: 160px 80px 100px 80px;
hero-left (Paso 7)	Flujo vertical; Ancho fijo 696; Espacio 32	flex-direction: column; · max-width: 696px; · gap: 32px;
Título (Paso 7)	Outfit ExtraBold 64, altura de línea 110 %	font-size: 64px; · font-weight: 800; · line-height: 1.1;
Botones (Paso 8)	Espacio 16; Espaciado 14 y 24; Radio 8; Geist SemiBold 15	gap: 16px; · padding: 14px 24px; · border-radius: var(--radius-sm);
hero-right (Paso 9)	Ancho fijo 520; Radio 12; Trazo #E5E7EB; Recortar contenido	max-width: 520px; · border-radius: var(--radius-md); · overflow: hidden;
Código (Paso 9)	Geist Mono 14, altura de línea 160 %; Espaciado 24	font-size: 14px; · line-height: 1.6; · padding: 24px;

Dos diferencias a propósito: en Figma, hero-left tiene Ancho fijo y hero-right Altura fija 404. En CSS usamos max-width y dejamos que el contenido decida la altura, para que el Hero pueda encogerse en pantallas pequeñas.

2.1 • El contenedor a dos columnas

```
/* 1. Contenedor flex a dos columnas con separación de 64px.
   Figma: hero-section, Flujo horizontal, Espacio 64 */
.hero-section {
  display: flex;                /* Disposición automática */
  align-items: center;          /* Alineación: centro vertical */
  justify-content: space-between;
  gap: 64px;                   /* Espacio 64 */
  padding: 160px 80px 100px 80px; /* Espaciado: arriba, derecha, abajo, izquierda */
  max-width: 1440px;           /* el ancho del marco de Figma */
  margin: 0 auto;              /* centra la caja en pantallas más anchas */
}
```

2.2 • La columna izquierda: un flex dentro de otro

```
/* 2. Columna izquierda • Figma: hero-left, Flujo vertical, Espacio 32 */
.hero-left {
  flex: 1;                                /* Llenar el contenedor */
  max-width: 696px;                       /* Ancho fijo 696, pero puede encogerse */
  display: flex;
  flex-direction: column;                 /* Flujo ↓ */
  gap: 32px;
}

.hero-title {
  font-size: 64px;
  font-weight: 800;
  color: var(--color-text-primary);
  line-height: 1.1;                      /* Altura de línea 110 % */
  letter-spacing: -0.02em;
}

.hero-description {
  font-size: 18px;
  color: var(--color-text-secondary);
  line-height: 1.6;                      /* Altura de línea 160 % */
}
```

`.hero-left` es a la vez un **ítem** de `.hero-section` (por eso lleva `flex: 1`) y un **contenedor** de su título, su párrafo y sus botones (por eso lleva `display: flex` y `flex-direction: column`). Es lo mismo que en Figma: un marco con Disposición automática dentro de otro.

Comprueba: la columna izquierda mide como máximo 696 px, y entre el título, el párrafo y los botones hay 32 px de separación.

2.3 • Los botones

```
/* 3. Botones de llamada a la acción • Figma: hero-actions, Espacio 16 */
.hero-actions {
  display: flex;
  gap: 16px;
  align-items: center;
}

.btn {
  display: inline-flex;                   /* el enlace deja de ser inline: acepta padding vertical */
  align-items: center;
  justify-content: center;
  padding: 14px 24px;                    /* Espaciado 14 y 24 */
  border-radius: var(--radius-sm);       /* Radio de esquina 8 */
  font-size: 15px;
  font-weight: 600;
}

.btn-primary {
  background-color: var(--color-primary);
  color: #FFFFFF;
  border: 1px solid var(--color-primary);
}

.btn-primary:hover {
  filter: brightness(0.92);
  box-shadow: 0 4px 14px rgba(6, 182, 196, 0.35);
}
```

```
.btn-secondary {
  background-color: transparent;
  color: var(--color-text-primary);
  border: 1px solid var(--color-text-secondary);
}

.btn-secondary:hover {
  background-color: rgba(11, 15, 25, 0.05);
}
```

Un enlace es inline: ignora el padding vertical al calcular su línea. Con `display: inline-flex` se comporta como una caja y además centra su texto. Los botones no llevan ancho: miden lo que su texto más el Espaciado, como **Ajustar al contenido** en Figma.

2.4 • La tarjeta de código

```
/* 4. Columna derecha: tarjeta de código • Figma: hero-right */
.hero-right {
  flex: 1;
  max-width: 520px; /* Ancho fijo 520 */
  display: flex;
  justify-content: center;
}

.code-card {
  width: 100%;
  background-color: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-md); /* Radio de esquina 12 */
  overflow: hidden; /* Recortar contenido */
  box-shadow: 0 10px 25px rgba(0, 0, 0, 0.05);
}

/* Figma: code-header, Flujo horizontal, Espacio Auto */
.code-header {
  display: flex;
  align-items: center;
  justify-content: space-between; /* Espacio: Auto */
  padding: 12px 16px;
  background-color: #F3F4F6;
  border-bottom: 1px solid var(--color-border);
}

/* Figma: window-controls, Espacio 8 */
.window-controls {
  display: flex;
  gap: 8px;
}

.dot {
  width: 10px;
  height: 10px;
  border-radius: 50%;
  display: inline-block; /* un span es inline: sin esto ignora width y height */
}

.dot-red { background-color: #EF4444; }
.dot-yellow { background-color: #F59E0B; }
.dot-green { background-color: #10B981; }

.code-filename {
  font-family: var(--font-code);
  font-size: 14px;
}
```

```

    color: var(--color-text-secondary);
}

/* Figma: code-body, Espaciado 24 */
.code-body {
  padding: 24px;
  font-family: var(--font-code);
  font-size: 14px;
  line-height: 1.6;
  background-color: #FFFFFF;
  margin: 0;
  border: none;
}

```

Cuenta los contenedores flex de la tarjeta: `.hero-right` centra la tarjeta, `.code-header` separa los círculos del nombre del archivo con `space-between` (el Espacio Auto de Figma) y `.window-controls` pone los tres círculos en fila con 8 px.

2.5 • El Hero en pantallas estrechas

Las media queries son tema de la sesión 12; por ahora copia este bloque al final de `index.css`. Fíjate en la idea: basta cambiar `flex-direction` a `column` para que las dos columnas se apilen.

```

/* 5. Anticipo de la sesión 12: el Hero en móviles y tablets */
@media (max-width: 992px) {
  .hero-section {
    flex-direction: column;          /* las dos columnas se apilan */
    padding: 60px 24px;
    text-align: center;
  }
  .hero-actions { justify-content: center; }
  .hero-right { width: 100%; }
  .code-body {
    text-align: left; /* el código no hereda el centrado del hero */
    overflow-x: auto; /* si una línea no cabe, se desplaza dentro de la tarjeta */
  }
}

@media (max-width: 600px) {
  .hero-title { font-size: 40px; }
}

```

Comprueba: el título y los botones quedan a la izquierda y la tarjeta de código a la derecha, centradas en vertical. Al estrechar la ventana, la tarjeta baja debajo del texto sin desbordar.

Paso 3 • El pie de página en columna (`assets/css/header.css`)

En Figma, `footer-wrapper` es un marco con Flujo vertical, Alineación al centro y Espacio 24. Reemplaza el bloque 5 de `header.css`:

```

/* 5. Pie de página común • Figma: footer-wrapper, Flujo vertical, Espacio 24 */
footer {
  display: flex;
  flex-direction: column;          /* Flujo ↓: el menú arriba, el texto abajo */
  align-items: center;             /* Alineación: centro */
  gap: 24px;                      /* Espacio 24: reemplaza al margin-bottom del menú */
  background-color: var(--color-surface);
  border-top: 1px solid var(--color-border);
  padding: 48px 80px;             /* Espaciado 48 y 80 */
  text-align: center;
}

```



```

    font-size: 13px;
    color: var(--color-text-secondary);
  }

  footer nav ul {
    display: flex;
    flex-wrap: wrap;                /* en un celular, los enlaces bajan de línea */
    justify-content: center;
    gap: 24px;
  }

  footer a:hover {
    color: var(--color-primary);
  }

```

El gap: 24px del pie reemplaza al margin-bottom: 24px que tenía el menú en la sesión 09: en un contenedor flex, la separación entre hijos la pone el padre. Y flex-wrap: wrap deja que los enlaces bajen de línea en un celular.

Paso 4 • Proyectos con Grid (mis-proyectos.html y contenido.css)

4.1 • La caja de 1440

Las dos columnas no caben en la caja de lectura de 800 px. En contenido.css, reemplaza el bloque 1 por este, con el ancho y el Espaciado de las secciones de Figma:

```

/* 1. Caja central: desde la sesión 10 mide lo mismo que el marco de Figma */
main {
  max-width: 1440px;          /* el ancho de mi-portafolio en Figma */
  margin: 0 auto;             /* auto: reparte el espacio sobrante y centra la caja */
  padding: 6.25rem 5rem;      /* 100px y 80px. En Figma: Espaciado 100 y 80 de las secciones */
}

/* Las líneas de lectura largas cansan: ningún párrafo suelto pasa de 70 caracteres */
main section > p {
  max-width: 70ch;            /* ch: el ancho del carácter «0» de la fuente */
}

```

La unidad ch mide el ancho del carácter «0» de la fuente: 70 caracteres por línea es una medida cómoda para leer. El selector main section > p solo afecta a los párrafos que son hijos directos de una sección.

4.2 • El marcado de las tarjetas

Copia las dos imágenes del paquete (proyecto-cinf.jpg y proyecto-certificados.jpg) en tu carpeta assets/img/, o usa capturas de tus propios proyectos de 1256 × 440 px. En mis-proyectos.html, reemplaza la sección de proyectos por esta:

```

<!-- SECCIÓN: Proyectos • Figma: projects-section -->
<section class="projects-section">
  <h2>Proyectos</h2>

  <!-- Cuadrícula de dos columnas • Figma: projects-grid -->
  <div class="projects-grid">
    <!-- Tarjeta • Figma: project-card -->
    <article class="project-card">
      
    <div class="project-text">
        <h3>Ecosistema CINF USS</h3>
        <p>
            Plataforma para la gestión académica y proyectos estudiantiles
            del Centro de Informática.
        </p>
    </div>
</article>

<article class="project-card">
    
    <div class="project-text">
        <h3>Generador de Certificados Web</h3>
        <p>
            Aplicación para automatizar la emisión y validación de
            certificados digitales.
        </p>
    </div>
</article>
</div>
</section>

```

4.3 • La rejilla

Crea al final de contenido.css el bloque 13 de maquetación, con este comentario de título y la rejilla de proyectos:

```

/* =====
13. Maquetación (sesión 10): Flexbox y Grid
Cada bloque sale de un marco con Disposición automática en Figma
===== */

/* 13.3 Proyectos • Figma: projects-grid, Espacio 24, dos tarjetas iguales */
.projects-grid {
    display: grid;
    grid-template-columns: repeat(2, 1fr); /* dos columnas de una fracción cada una */
    gap: 24px;
}

/* Figma: project-card, Espacio 0, Espaciado 0, Recortar contenido */
.project-card {
    padding: 0; /* anula el 1.5rem de las tarjetas: la imagen va de borde a borde */
    margin-bottom: 0; /* en la rejilla, la separación la pone gap */
    overflow: hidden; /* Recortar contenido: la imagen respeta las esquinas */
}

.project-image {
    display: block; /* una imagen es inline: deja un hueco debajo si no se cambia */
    width: 100%;
    height: 220px; /* Figma: rectángulo de 220 de alto */
    object-fit: cover; /* llena la caja sin deformarse, como el modo Llenar de Figma */
    border-radius: 0; /* la clase gana a la regla de img: las esquinas las recorta la
tarjeta */
}

```

```

}

.project-text {
  padding: 24px;          /* Figma: project-text, Espaciado 24 */
}

```

Propiedad o valor	Qué hace
display: grid	La caja ordena a sus hijos en filas y columnas
grid-template-columns: repeat(2, 1fr)	Dos columnas. fr es una fracción del espacio libre: dos de 1fr miden lo mismo
gap: 24px	La separación entre columnas y entre filas: el Espacio 24 de projects-grid
overflow: hidden	Recortar contenido: la imagen no se sale por las esquinas redondeadas de la tarjeta
object-fit: cover	La imagen llena su caja de 220 de alto sin deformarse; lo que sobra se recorta
display: block en la imagen	Una imagen es inline y deja un hueco de unos píxeles debajo, el espacio de la línea de texto

Especificidad en acción: en la sesión 09 escribiste `img { border-radius: var(--radius-md); }` y `article { padding: 1.5rem; }`. `.project-image` y `.project-card` son clases (0-1-0) y ganan a esas reglas de tipo (0-0-1): por eso `border-radius: 0` y `padding: 0` se aplican.

Comprueba: las dos tarjetas miden lo mismo, la imagen va de borde a borde y respeta las esquinas redondeadas, y entre las tarjetas hay 24 px.

Paso 5 • Sobre mí a dos columnas con Flexbox

En Figma, `about-content` pone el texto a la izquierda y el gráfico de 480×264 a la derecha, con Espacio 64. En `sobre-mi.html`, envuelve la foto y el párrafo en `about-text`, y ese bloque y el aside en `about-content`:

```

<!-- SECCIÓN: Sobre mí • Figma: about-section -->
<section class="about-section">
  <h2>Sobre mí</h2>

  <!-- Dos columnas • Figma: about-content -->
  <div class="about-content">
    <!-- Columna izquierda • Figma: about-text -->
    <div class="about-text">
      <figure>
        
        <figcaption>Presentación al Banco BBVA</figcaption>
      </figure>

      <p>

```

```

        Hola, soy Dagner Chuman. Docente e Ingeniero enfocado en el
        desarrollo web y la formación tecnológica. Este es mi portafolio
        personal construido con HTML5 y estructura semántica limpia.
    </p>
</div>

<!-- Columna derecha · Figma: about-graphic -->
<aside aria-label="Datos rápidos">
    <h2>En resumen</h2>
    <dl>
        <dt>Institución:</dt>
        <dd>Centro de Informática USS</dd>
        <dt>Profesión:</dt>
        <dd>Docente e Ingeniero</dd>
        <dt>Especialidad:</dt>
        <dd>Desarrollo web y formación tecnológica</dd>
        <dt>Habilidades:</dt>
        <dd>HTML5, CSS3, Java</dd>
    </dl>
</aside>
</div>
</section>

```

Añade el bloque 13.1 en contenido.css, antes del 13.3:

```

/* 13.1 Sobre mí · Figma: about-content, Flujo →, Espacio 64, centro vertical */
.about-content {
    display: flex;
    align-items: center;
    gap: 64px;
}

.about-text {
    flex: 1;
    /* Llenar el contenedor: toma el espacio que sobra */
}

.about-content aside {
    flex: 0 0 480px;
    /* no crece, no se encoge, mide 480. Figma: Ancho fijo 480 */
    margin-bottom: 0;
}

```

Comprueba: «En resumen» queda a la derecha con 480 px de ancho, centrado en vertical respecto de la foto y el párrafo.

Paso 6 · Contacto: Flexbox en la lista y Grid en el formulario

6.1 · Las tarjetas de contacto

En contactame.html, reemplaza la primera sección por esta:

```

<!-- SECCIÓN: Contacto · Figma: contact-section -->
<section class="contact-section">
    <!-- Dos lados · Figma: contact-content -->
    <div class="contact-content">
        <!-- Lado izquierdo · Figma: contact-text -->
        <div class="contact-text">
            <h2>Contacto</h2>
            <p>Puedes contactarme a través de los siguientes canales:</p>
        </div>

        <!-- Lado derecho · Figma: contact-list y tres contact-card -->
        <ul class="contact-list">

```

```

<li class="contact-card">
  <span class="contact-icon" aria-hidden="true">@</span>
  <span>
    <strong>Correo</strong>
    <a href="mailto:dchuman@uss.edu.pe">dchuman@uss.edu.pe</a>
  </span>
</li>
<li class="contact-card">
  <span class="contact-icon" aria-hidden="true">in</span>
  <span>
    <strong>LinkedIn</strong>
    <a href="https://www.linkedin.com" target="_blank"
      >linkedin.com/in/dagnerchuman</a>
  >
  </span>
</li>
<li class="contact-card">
  <span class="contact-icon" aria-hidden="true">GH</span>
  <span>
    <strong>GitHub</strong>
    <a href="https://github.com" target="_blank"
      >github.com/dagnerchuman</a>
  >
  </span>
</li>
</ul>
</div>
</section>

```

Y en contenido.css, después del 13.3:

```

/* 13.4 Contacto · Figma: contact-content, Flujo →, Espacio 64 */
.contact-content {
  display: flex;
  align-items: flex-start;
  gap: 64px;
}

.contact-text {
  flex: 1;
}

/* Figma: contact-list, Flujo ↓, Ancho fijo 520, Espacio 16 */
.contact-list {
  flex: 0 0 520px;
  display: flex;
  flex-direction: column;
  gap: 16px;
}

/* Figma: contact-card, Flujo →, Espaciado 20, Espacio 16, Alineación centro */
.contact-card {
  display: flex;
  align-items: center;
  gap: 16px;
  padding: 20px;
  margin-bottom: 0; /* la clase gana a «main li» */
  background-color: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-sm);
}

/* El círculo de 44 × 44: flex también sirve para centrar en los dos ejes */
.contact-icon {

```

```

    flex-shrink: 0;          /* nunca se aplasta aunque el texto sea largo */
    width: 44px;
    height: 44px;
    border-radius: 50%;
    display: flex;
    align-items: center;
    justify-content: center;
    background-color: var(--color-primary);
    color: #FFFFFF;
    font-weight: 700;
    font-size: 0.875rem;
  }

  .contact-card strong,
  .contact-card a {
    display: block;          /* de inline a block: cada una en su propia línea */
  }

  .contact-card strong {
    font-size: 0.75rem;
    text-transform: uppercase;
    letter-spacing: 0.06em;
    color: var(--color-text-secondary);
  }
}

```

El truco para centrar: un contenedor con `display: flex`, `align-items: center` y `justify-content: center` centra a su hijo en los dos ejes. Así quedan las letras dentro del círculo de 44 × 44.

6.2 • El formulario en rejilla

En el mismo archivo HTML, añade la clase `form-grid` al `<form>` y la clase `form-actions` al párrafo de los botones:

```

<form class="form-grid" action="contactame.html" method="get">
  <fieldset> ... Tus datos ... </fieldset>
  <fieldset> ... Tu mensaje ... </fieldset>

  <!-- Ocupa las dos columnas de la rejilla -->
  <p class="form-actions">
    <button type="submit">Enviar mensaje</button>
    <button type="reset">Limpiar</button>
  </p>
</form>

```

En `contenido.css`, después del 13.4:

```

/* 13.5 Formulario: una rejilla de dos columnas, un fieldset en cada una */
.form-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 24px;
  align-items: start;
}

.form-grid fieldset {
  margin-bottom: 0;
}

.form-actions {
  grid-column: 1 / -1;      /* de la primera línea a la última: ocupa las dos columnas */
  display: flex;
  gap: 16px;
}

```

grid-column: 1 / -1 se lee «desde la primera línea de la rejilla hasta la última»: el párrafo de los botones ocupa las dos columnas. Las líneas de una rejilla de dos columnas son tres: 1, 2 y 3; el -1 es siempre la última.

Comprueba: a la derecha del título hay tres filas iguales con un círculo azul; debajo, «Tus datos» y «Tu mensaje» quedan lado a lado y los botones ocupan todo el ancho.

Paso 7 • La multimedia en rejilla, las pantallas estrechas y la publicación

7.1 • Una columna fija y otra flexible

En sobre-mi.html, dentro de la sección «Mi presentación», envuelve el contenido en dos niveles: un div con la clase media-grid y, dentro, la figura del video y un div con todo lo demás:

```

<div class="media-grid">
  <figure> ... video propio, controles y figcaption ... </figure>

  <div>
    <figure> ... iframe de YouTube ... </figure>
    <figure> ... audio ... </figure>
    <details> ... transcripción ... </details>
  </div>
</div>

```

Y en contenido.css, el bloque 13.2, después del 13.1:

```

/* 13.2 Multimedia: una columna fija para el video vertical y otra flexible */
.media-grid {
  display: grid;
  grid-template-columns: 315px 1fr; /* 315px fijos + todo lo que sobra */
  gap: 64px;
  align-items: start;
}

```

315px 1fr mezcla una medida fija con una fracción: la primera columna mide lo que el video vertical y la segunda toma todo el espacio que queda.

7.2 • Todo a una columna en pantallas estrechas

Copia este bloque al final de contenido.css. En la sesión 12 escribirás tus propias media queries:

```

/* 14. Anticipo de la sesión 12: en pantallas estrechas todo pasa a una columna */
@media (max-width: 992px) {
  main {
    padding: 3.75rem 1.5rem;          /* 60px y 24px */
  }
  .about-content,
  .contact-content {
    flex-direction: column;          /* Flujo → pasa a Flujo ↓ */
    align-items: stretch;
  }
  .about-content aside,
  .contact-list {
    flex-basis: auto;                /* deja de medir 480 o 520 */
  }
  .media-grid,
  .projects-grid,
  .form-grid {
    grid-template-columns: 1fr;      /* una sola columna */
  }
}

```

7.3 • Valida y publica

1. Valida las cuatro páginas en **validator.w3.org** y tus cuatro hojas de estilo en **jigsaw.w3.org/css-validator**: cero errores.
2. Estrecha la ventana del navegador hasta el ancho de un celular: no debe aparecer barra de desplazamiento horizontal.
3. Publica en Netlify y abre la URL en tu celular.

Lista de comprobación

- ☐ `index.html` enlaza `global.css`, `header.css` e `index.css`, en ese orden.
- ☐ El Hero tiene dos columnas centradas en vertical, con 64 px entre ellas y el Espaciado 160 · 80 · 100 · 80.
- ☐ Cada contenedor flex o grid del CSS corresponde a un marco con Disposición automática en tu Figma.
- ☐ Las tarjetas de proyecto miden lo mismo y la imagen respeta sus esquinas.
- ☐ Sobre mí y Contacto tienen dos columnas en escritorio y una en el celular.
- ☐ Los botones del formulario ocupan las dos columnas de la rejilla.
- ☐ Ninguna página tiene desplazamiento horizontal al estrecharla.
- ☐ El HTML y el CSS pasan los validadores del W3C sin errores.

Errores frecuentes en esta sesión

Síntoma	Causa y solución
display: flex no hace nada	Está en el hijo y no en el padre. Flexbox y Grid se escriben en el contenedor; los hijos directos son los que se ordenan
Un elemento no se mueve con justify-content	Ese eje no es el que crees. Con <code>flex-direction: column</code> , <code>justify-content</code> trabaja en vertical y <code>align-items</code> en horizontal
Las dos columnas no miden lo que deberían	Falta <code>flex: 1</code> en la columna que debe llenar el espacio, o <code>flex: 0 0 480px</code> en la de ancho fijo
La imagen deja un hueco debajo	Falta <code>display: block;</code> en <code>.project-image</code>

Síntoma	Causa y solución
La imagen sale deformada	Falta <code>object-fit: cover;</code> o se escribió <code>height</code> sin <code>width: 100%</code>
La imagen tapa las esquinas redondeadas	Falta <code>overflow: hidden;</code> en <code>.project-card</code>
Los botones del formulario quedan en una sola columna	Falta la clase <code>form-actions</code> en el párrafo o <code>grid-column: 1 / -1;</code> en su regla
Aparece desplazamiento horizontal en el celular	Falta el bloque 14 (media queries) o un ancho fijo en px no tiene su <code>max-width: 100%</code>